

Manual de Usuario

Simulador de Decisiones para Plataforma de Streaming

Universidad Rafael Landívar · Pensamiento Computacional

1. Información general

Nombre del sistema: Simulador de Decisiones para Plataforma de Streaming

Objetivo: Ayudar al equipo editorial a evaluar si un contenido puede publicarse en la plataforma, aplicando reglas de clasificación, duración, horario y nivel de producción.

Lenguaje: C#

Interfaz: Consola de texto

2. Cómo ejecutar el programa

1. Tener instalado .NET SDK 6.0 o superior.
2. Abrir la terminal y entrar a la carpeta del proyecto.
3. Escribir el siguiente comando y presionar ENTER:

```
dotnet run
```
4. El menú principal aparecerá en pantalla.

3. Opciones del menú

Al iniciar el programa aparece el siguiente menú:

```
=== SIMULADOR DE STREAMING === 1. Evaluar nuevo contenido 2. Mostrar reglas  
del sistema 3. Mostrar estadísticas 4. Reiniciar estadísticas 5. Salir
```

Opción	Descripción
1. Evaluar	Ingresa los datos de un contenido y el sistema da una decisión.
2. Reglas	Muestra las reglas que usa el sistema para evaluar.

3. Estadísticas	Muestra los totales de la sesión: evaluados, publicados, rechazados, en revisión y porcentaje de aprobación.
4. Reiniciar	Pone en cero todos los contadores. Pide confirmación.
5. Salir	Cierra el programa y muestra el resumen final.

4. Funcionamiento del sistema

Al elegir la opción 1, el sistema pide los siguientes datos:

- Tipo de contenido: Película, Serie, Documental o Evento en vivo
- Duración en minutos
- Clasificación: Todo público, +13 o +18
- Hora programada (0 a 23)
- Nivel de producción: Bajo, Medio o Alto

Validación técnica

El sistema verifica tres reglas obligatorias. Si alguna falla, el contenido es rechazado de inmediato.

Tipo	Duración permitida	Horario por clasificación
Película	60 – 180 min	Todo público: cualquier hora
Serie	20 – 90 min	+13: entre 6:00 y 22:00
Documental	30 – 120 min	+18: entre 22:00 y 5:00
Evento en vivo	30 – 240 min	

Regla de producción: la producción baja solo es válida para contenidos Todo público o +13. La producción media o alta es válida para cualquier clasificación.

Clasificación de impacto

Si el contenido pasa la validación técnica, el sistema determina su nivel de impacto:

- Alto: producción alta, duración mayor a 120 min, o programado entre las 20:00 y 23:00.
- Medio: producción media, o duración entre 60 y 120 min.
- Bajo: producción baja y duración menor a 60 min.

Si aplica más de un nivel, se toma el más alto.

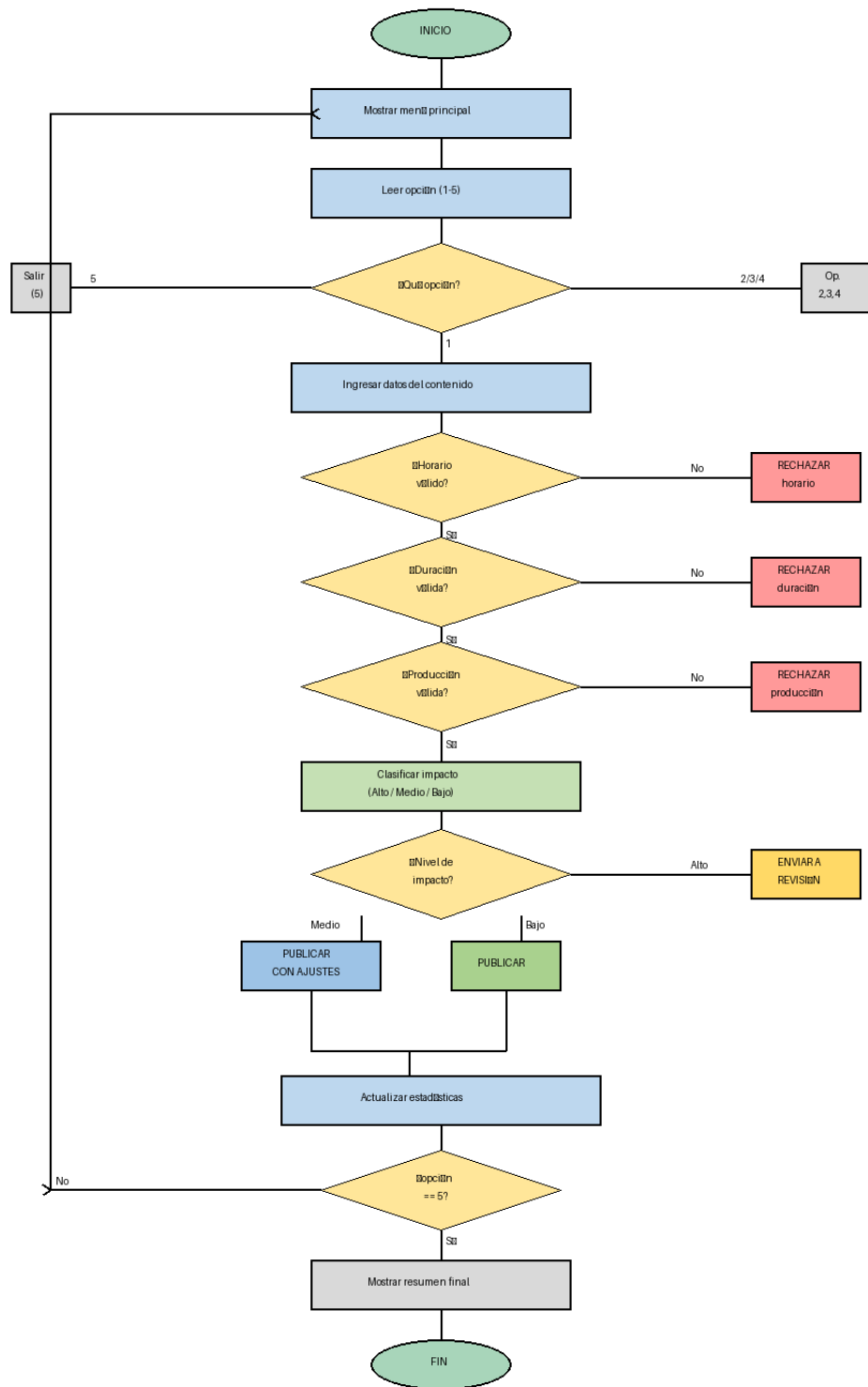
Decisión final

Decisión	Cuándo ocurre
PUBLICAR	Cumple todas las reglas e impacto Bajo.

PUBLICAR CON AJUSTES	Cumple todas las reglas e impacto Medio.
ENVIAR A REVISIÓN	Cumple las reglas pero tiene impacto Alto.
RECHAZAR	Incumple alguna regla obligatoria.

5. Diagrama de flujo

El siguiente diagrama representa el flujo completo del programa, desde que inicia hasta que el usuario decide salir.



6. Ejemplo paso a paso

Datos ingresados:

- Tipo: Serie
- Duración: 45 minutos
- Clasificación: Todo público
- Hora: 15
- Producción: Media

Lo que muestra el programa:

--- RESULTADO --- Decision: PUBLICAR CON AJUSTES Razon: Cumple reglas técnicas. Impacto medio. Impacto: Medio

Por qué ese resultado:

- 45 min está dentro del rango de Serie (20–90). ✓
- Todo público puede programarse a cualquier hora. ✓
- Producción media es válida para cualquier clasificación. ✓
- Impacto Medio porque la producción es media → decisión: PUBLICAR CON AJUSTES.

7. Estadísticas de la sesión

La opción 3 muestra los siguientes datos acumulados desde que se inició el programa:

- Total de contenidos evaluados
- Cantidad publicados
- Cantidad rechazados
- Cantidad enviados a revisión
- Impacto predominante (el nivel que más veces se asignó)
- Porcentaje de aprobación

Estos datos se reinician con la opción 4 o al cerrar el programa.

8. Pruebas realizadas

Se quiso realizar de nuevo el uso del sistema pero saltaron estas líneas de error

Program.cs | Explorador de soluciones | ConsoleApp127 | StreamingSimulator | totalEvaluados

```
91 if (pasa && ValidarDuracion(tipo, duracion))
92 {
93     pasa = false;
94     razonRechazo = "Duración (duracion) sin fuera del rango permitido para (tipoTeste).";
95 }
96 if (pasa && ValidarProduccion(produccion, clasif))
97 {
98     pasa = false;
99     razonRechazo = "Producción (produccionTeste) no válida para clasificación (clasificTeste).";
100 }
101 string impacto = "";
102 if (pasa)
103 {
104     impacto = ClasificarImpacto(produccion, duracion, hora);
105     Console.WriteLine($" Impacto: {impacto}");
106 }
107 string decision = "";
108 string razon = "";
109 if (!pasa)
110 {
111     decision = "RECHAZADO";
112     razon = razonRechazo;
113 }
114 else if (impacto == "Alto")
115 {
116     decision = "TRANSICIÓN A REVISIÓN";
117     razon = "El contenido tiene Impacto Alto.";
118 }
119 else if (impacto == "Bajo") || impacto == "Medio"
120 {
121     bool necesidadAjuste = VerificarAjusteRequisitos(tipo, duracion, clasif, hora);
122     if (necesidadAjuste)
123     {
124         decision = "PUBLICAR CON AJUSTES";
125         razon = "Cumple reglas técnicas pero requiere ajuste menor de horario o duración.";
126     }
127     else
128     {
129         decision = "PUBLICAR";
130         razon = "Cumple todas las reglas técnicas con impacto " + impacto + ".";
131     }
132 }
133 }
134
135 Console.WriteLine($" DECISIÓN FINAL: {decision}");
136 Console.WriteLine($" Razon: {razon}");
137 Console.WriteLine("");
138 ActualizarEstadisticas(decision, impacto);
139 Console.WriteLine("Apresione ENTER para continuar...");
140 Console.ReadKey();
141 }
```

49 % | Más de 99 | 0 | 0 | 0 | Línea: 1, Carácter: 14 | SPC | CRLF | UTF-8 with BOM

Lista de errores

Toda la solución | 128 Errores | 0 Advertencias | 0 de 10 Mensajes | Compilación + IntelliSense | Lista de errores de búsqueda

Cód...	Descripción	Proyecto	Archivo	
CS0101	El espacio de nombres '<global namespace>' ya contiene una definición para 'StreamingSimulator'	ConsoleApp127	Proyecto 1 PC.cs	2
CS0101	El espacio de nombres '<global namespace>' ya contiene una definición para 'StreamingSimulator'	ConsoleApp127	Program.cs	3
CS0111	El tipo 'StreamingSimulator' ya define un miembro denominado 'Main' con los mismos tipos de parámetro	ConsoleApp127	Proyecto 1 PC.cs	12
CS0111	El tipo 'StreamingSimulator' ya define un miembro denominado 'Main' con los mismos tipos de parámetro	ConsoleApp127	Program.cs	13

Captura de su funcionamiento hoy 25/3/2026

SIMULADOR DE DECISIONES – PLATAFORMA DE STREAMING

Universidad Rafael Landívar · Pensamiento Computacional

MENÚ PRINCIPAL

1. Evaluar nuevo contenido
 2. Mostrar reglas del sistema
 3. Mostrar estadísticas de la sesión
 4. Reiniciar estadísticas
 5. Salir
-

Selecciona una opción (1-5): |

Selecciona una opción (1-5): 1

EVALUACIÓN DE NUEVO CONTENIDO

Tipo de contenido:

1. Película 2. Serie 3. Documental 4. Evento en vivo

Selecciona (1-4):

RESULTADO DE LA EVALUACIÓN

Tipo : Serie
Duración : 260 min
Clasificación: Todo público
Hora : 22:00
Producción : Media

DECISIÓN FINAL: RECHAZAR

Razón: Duración 260 min fuera del rango permitido para Serie.

Presiona ENTER para continuar...

- Alta : válida para cualquier clasificación

► IMPACTO

- Alto : producción alta, duración > 120 min, o hora 20-23
- Medio: producción media, o duración 60-120 min
- Bajo : producción baja y duración < 60 min

► DECISIÓN FINAL

- Publicar : todas las reglas + impacto Bajo/Medio
- Publicar con ajustes: reglas OK + valor en borde de rango
- Enviar a revisión : reglas OK + impacto Alto
- Rechazar : incumple alguna regla obligatoria

Presiona ENTER para continuar...

|

MENÚ PRINCIPAL

-
1. Evaluar nuevo contenido
 2. Mostrar reglas del sistema
 3. Mostrar estadísticas de la sesión
 4. Reiniciar estadísticas
 5. Salir
-

Selecciona una opción (1-5): 3

ESTADÍSTICAS DE LA SESIÓN

Total evaluados	: 1
Publicados	: 0 (incl. con ajustes: 0)
Rechazados	: 1
En revisión	: 0
Porcentaje de aprobación	: 0%
Impacto predominante	: N/A

Presiona ENTER para continuar...

MENÚ PRINCIPAL

1. Evaluar nuevo contenido
 2. Mostrar reglas del sistema
 3. Mostrar estadísticas de la sesión
 4. Reiniciar estadísticas
 5. Salir
-

Selecciona una opción (1-5): 4

¿Confirmar reinicio de estadísticas? (s/n):

MENÚ PRINCIPAL

1. Evaluar nuevo contenido
 2. Mostrar reglas del sistema
 3. Mostrar estadísticas de la sesión
 4. Reiniciar estadísticas
 5. Salir
-

Selecciona una opción (1-5): 5

RESUMEN FINAL DE LA SESIÓN

Total evaluados	:	0
Publicados	:	0
Rechazados	:	0
En revisión	:	0
Aprobación	:	0%

¡Hasta luego!

```

CODIGO
using System;

class StreamingSimulator
{
    static int totalEvaluados = 0;
    static int totalPublicados = 0;
    static int totalRechazados = 0;
    static int totalRevision = 0;
    static int totalPublicadosConAjustes = 0;
    static int impactoAltoCount = 0;
    static int impactoMedioCount = 0;
    static int impactoBajoCount = 0;
    static void Main()
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;
        MostrarBienvenida();

        int opcion;
        do
        {
            MostrarMenu();
            opcion = LeerEnteroValido("Selecciona una opción (1-5): ", 1, 5);

            switch (opcion)
            {
                case 1: EvaluarContenido(); break;
                case 2: MostrarReglas(); break;
                case 3: MostrarEstadisticas(); break;
                case 4: ReiniciarEstadisticas(); break;
                case 5: /* salir */ break;
                default: Console.WriteLine("Opción no válida."); break;
            }

        } while (opcion != 5);

        MostrarResumenFinal();
        Console.WriteLine("\n¡Hasta luego!\n");
    }
    static void MostrarBienvenida()
    {
        Console.Clear();
        Linea('=', 60);
        Console.WriteLine("  SIMULADOR DE DECISIONES — PLATAFORMA DE  
STREAMING");
        Linea('=', 60);
        Console.WriteLine("  Universidad Rafael Landívar · Pensamiento Computacional");
        Linea('-', 60);
        Console.WriteLine();
    }
}

```

```

static void MostrarMenu()
{
    Console.WriteLine();
    Linea('-', 60);
    Console.WriteLine("                MENÚ PRINCIPAL");
    Linea('-', 60);
    Console.WriteLine("  1. Evaluar nuevo contenido");
    Console.WriteLine("  2. Mostrar reglas del sistema");
    Console.WriteLine("  3. Mostrar estadísticas de la sesión");
    Console.WriteLine("  4. Reiniciar estadísticas");
    Console.WriteLine("  5. Salir");
    Linea('-', 60);
}

static void EvaluarContenido()
{
    Console.WriteLine();
    Linea('-', 60);
    Console.WriteLine("          EVALUACIÓN DE NUEVO CONTENIDO");
    Linea('-', 60);
    int tipo = LeerTipoContenido();
    int duracion = LeerEnteroValido("Duración en minutos: ", 1, 9999);
    int clasif = LeerClasificacion();
    int hora = LeerEnteroValido("Hora programada (0-23): ", 0, 23);
    int produccion = LeerNivelProduccion();
    string tipoTexto = ObtenerTextoTipo(tipo);
    string clasifTexto = ObtenerTextoClasif(clasif);
    string produccionTexto = ObtenerTextoProduccion(produccion);
    Console.WriteLine();
    Linea('-', 60);
    Console.WriteLine(" RESULTADO DE LA EVALUACIÓN");
    Linea('-', 60);
    Console.WriteLine($" Tipo      : {tipoTexto}");
    Console.WriteLine($" Duración   : {duracion} min");
    Console.WriteLine($" Clasificación: {clasifTexto}");
    Console.WriteLine($" Hora       : {hora}:00");
    Console.WriteLine($" Producción  : {produccionTexto}");
    Linea('-', 60);
    bool pasa = true;
    string razonRechazo = "";
    if (!ValidarHorario(clasif, hora))
    {
        pasa = false;
        razonRechazo = $"Horario {hora}:00 no permitido para clasificación {clasifTexto}.";
    }
    if (pasa && !ValidarDuracion(tipo, duracion))
    {
        pasa = false;
        razonRechazo = $"Duración {duracion} min fuera del rango permitido para {tipoTexto}.";
    }
    if (pasa && !ValidarProduccion(produccion, clasif))
    {

```

```

        pasa = false;
        razonRechazo = $"Producción {produccionTexto} no válida para clasificación
{clasificTexto}.";
    }
    string impacto = "";
    if (pasa)
    {
        impacto = ClasificarImpacto(produccion, duracion, hora);
        Console.WriteLine($" Impacto : {impacto}");
    }
    string decision = "";
    string razon = "";
    if (!pasa)
    {
        decision = "RECHAZAR";
        razon = razonRechazo;
    }
    else if (impacto == "Alto")
    {
        decision = "ENVIAR A REVISIÓN";
        razon = "El contenido tiene impacto Alto.";
    }
    else if (impacto == "Bajo" || impacto == "Medio")
    {
        bool necesitaAjuste = VerificarAjusteMenor(tipo, duracion, clasif, hora);
        if (necesitaAjuste)
        {
            decision = "PUBLICAR CON AJUSTES";
            razon = "Cumple reglas técnicas pero requiere ajuste menor de horario o duración.";
        }
        else
        {
            decision = "PUBLICAR";
            razon = "Cumple todas las reglas técnicas con impacto " + impacto + ".";
        }
    }
}

Linea('=', 60);
Console.WriteLine($" DECISIÓN FINAL: {decision}");
Console.WriteLine($" Razón: {razon}");
Linea('=', 60);
ActualizarEstadisticas(decision, impacto);

Console.WriteLine("\nPresiona ENTER para continuar...");
Console.ReadLine();
}

static bool ValidarHorario(int clasif, int hora)
{
    if (clasif == 1) return true;
    if (clasif == 2) return hora >= 6 && hora <= 22;
    if (clasif == 3) return hora >= 22 || hora <= 5;
}

```

```

        return false;
    }

    static bool ValidarDuracion(int tipo, int duracion)
    {
        if (tipo == 1) return duracion >= 60 && duracion <= 180;
        if (tipo == 2) return duracion >= 20 && duracion <= 90;
        if (tipo == 3) return duracion >= 30 && duracion <= 120;
        if (tipo == 4) return duracion >= 30 && duracion <= 240;
        return false;
    }

    static bool ValidarProduccion(int produccion, int clasif)
    {
        if (produccion == 1) return clasif == 1 || clasif == 2;
        return true;
    }

    static string ClasificarImpacto(int produccion, int duracion, int hora)
    {
        bool esAlto = (produccion == 3) || (duracion > 120) || (hora >= 20 && hora <= 23);
        bool esMedio = (produccion == 2) || (duracion >= 60 && duracion <= 120);
        bool esBajo = (produccion == 1) && (duracion < 60);
        if (esAlto) return "Alto";
        if (esMedio) return "Medio";
        return "Bajo";
    }

    static bool VerificarAjusteMenor(int tipo, int duracion, int clasif, int hora)
    {
        if (clasif == 2 && (hora == 6 || hora == 22)) return true;
        if (clasif == 3 && (hora == 22 || hora == 5)) return true;
        if (tipo == 1 && (duracion == 60 || duracion == 180)) return true;
        if (tipo == 2 && (duracion == 20 || duracion == 90)) return true;
        if (tipo == 3 && (duracion == 30 || duracion == 120)) return true;
        if (tipo == 4 && (duracion == 30 || duracion == 240)) return true;

        return false;
    }

    static void ActualizarEstadisticas(string decision, string impacto)
    {
        totalEvaluados++;

        if (decision == "PUBLICAR") totalPublicados++;
        else if (decision == "PUBLICAR CON AJUSTES") { totalPublicados++;
totalPublicadosConAjustes++; }
        else if (decision == "RECHAZAR") totalRechazados++;
        else if (decision == "ENVIAR A REVISIÓN") totalRevision++;

        if (impacto == "Alto") impactoAltoCount++;
        else if (impacto == "Medio") impactoMedioCount++;
        else if (impacto == "Bajo") impactoBajoCount++;
    }

```

```

static void MostrarReglas()
{
    Console.WriteLine();
    Linea('═', 60);
    Console.WriteLine("          REGLAS DEL SISTEMA");
    Linea('═', 60);

    Console.WriteLine("\n ► CLASIFICACIÓN Y HORARIO");
    Console.WriteLine(" • Todo público : cualquier hora");
    Console.WriteLine(" • +13      : entre 6:00 y 22:00");
    Console.WriteLine(" • +18      : entre 22:00 y 5:00");

    Console.WriteLine("\n ► DURACIÓN POR TIPO");
    Console.WriteLine(" • Película   : 60 – 180 min");
    Console.WriteLine(" • Serie      : 20 – 90 min");
    Console.WriteLine(" • Documental : 30 – 120 min");
    Console.WriteLine(" • Evento en vivo: 30 – 240 min");

    Console.WriteLine("\n ► PRODUCCIÓN");
    Console.WriteLine(" • Baja : solo válida para Todo público o +13");
    Console.WriteLine(" • Media : válida para cualquier clasificación");
    Console.WriteLine(" • Alta : válida para cualquier clasificación");

    Console.WriteLine("\n ► IMPACTO");
    Console.WriteLine(" • Alto : producción alta, duración > 120 min, o hora 20–23");
    Console.WriteLine(" • Medio: producción media, o duración 60–120 min");
    Console.WriteLine(" • Bajo : producción baja y duración < 60 min");

    Console.WriteLine("\n ► DECISIÓN FINAL");
    Console.WriteLine(" • Publicar      : todas las reglas + impacto Bajo/Medio");
    Console.WriteLine(" • Publicar con ajustes: reglas OK + valor en borde de rango");
    Console.WriteLine(" • Enviar a revisión : reglas OK + impacto Alto");
    Console.WriteLine(" • Rechazar      : incumple alguna regla obligatoria");

    Linea('═', 60);
    Console.WriteLine("\nPresiona ENTER para continuar...");
    Console.ReadLine();
}

static void MostrarEstadisticas()
{
    Console.WriteLine();
    Linea('═', 60);
    Console.WriteLine("          ESTADÍSTICAS DE LA SESIÓN");
    Linea('═', 60);

    Console.WriteLine($" Total evaluados      : {totalEvaluados}");
    Console.WriteLine($" Publicados          : {totalPublicados} (incl. con ajustes: {totalPublicadosConAjustes})");
    Console.WriteLine($" Rechazados          : {totalRechazados}");
    Console.WriteLine($" En revisión         : {totalRevision}");
    double porcentaje = totalEvaluados > 0

```

```

        ? Math.Round((double)totalPublicados / totalEvaluados * 100, 1)
        : 0;
Console.WriteLine($" Porcentaje de aprobación : {porcentaje}%");
string predominante = "N/A";
int maxImpacto = 0;
string[] nombresImpacto = { "Alto", "Medio", "Bajo" };
int[] valoresImpacto = { impactoAltoCount, impactoMedioCount, impactoBajoCount };

for (int i = 0; i < 3; i++)
{
    if (valoresImpacto[i] > maxImpacto)
    {
        maxImpacto = valoresImpacto[i];
        predominante = nombresImpacto[i];
    }
}
Console.WriteLine($" Impacto predominante : {predominante}");

Linea('═', 60);
Console.WriteLine("\nPresiona ENTER para continuar...");
Console.ReadLine();
}
static void ReiniciarEstadisticas()
{
    Console.WriteLine("\n ¿Confirmar reinicio de estadísticas? (s/n): ");
    string resp = Console.ReadLine()?.Trim().ToLower() ?? "n";
    if (resp == "s")
    {
        totalEvaluados = totalPublicados = totalRechazados = totalRevision = 0;
        totalPublicadosConAjustes = 0;
        impactoAltoCount = impactoMedioCount = impactoBajoCount = 0;
        Console.WriteLine(" Estadísticas reiniciadas correctamente.");
    }
    else
    {
        Console.WriteLine(" Operación cancelada.");
    }
    Console.WriteLine("\nPresiona ENTER para continuar...");
    Console.ReadLine();
}
static void MostrarResumenFinal()
{
    Console.WriteLine();
    Linea('═', 60);
    Console.WriteLine(" RESUMEN FINAL DE LA SESIÓN");
    Linea('═', 60);
    Console.WriteLine($" Total evaluados : {totalEvaluados}");
    Console.WriteLine($" Publicados : {totalPublicados}");
    Console.WriteLine($" Rechazados : {totalRechazados}");
    Console.WriteLine($" En revisión : {totalRevision}");
    double porcentaje = totalEvaluados > 0

```



```

        ? Math.Round((double)totalPublicados / totalEvaluados * 100, 1)
        : 0;
    Console.WriteLine($" Aprobación      : {porcentaje}%");
    Linea('=', 60);
}
/// <summary>Lee un entero dentro del rango [min, max], reintentando si la entrada no es
válida.</summary>
static int LeerEnteroValido(string mensaje, int min, int max)
{
    int valor;
    bool valido;
    do
    {
        Console.Write(" " + mensaje);
        valido = int.TryParse(Console.ReadLine(), out valor);
        if (!valido || valor < min || valor > max)
        {
            Console.WriteLine($" ⚠ Valor inválido. Ingresa un número entre {min} y {max}.");
            valido = false;
        }
    } while (!valido);
    return valor;
}

static int LeerTipoContenido()
{
    Console.WriteLine(" Tipo de contenido:");
    Console.WriteLine(" 1. Película  2. Serie  3. Documental  4. Evento en vivo");
    return LeerEnteroValido("Selecciona (1-4): ", 1, 4);
}

static int LeerClasificacion()
{
    Console.WriteLine(" Clasificación:");
    Console.WriteLine(" 1. Todo público  2. +13  3. +18");
    return LeerEnteroValido("Selecciona (1-3): ", 1, 3);
}

static int LeerNivelProduccion()
{
    Console.WriteLine(" Nivel de producción:");
    Console.WriteLine(" 1. Bajo  2. Medio  3. Alto");
    return LeerEnteroValido("Selecciona (1-3): ", 1, 3);
}

static string ObtenerTextoTipo(int tipo)
{
    if (tipo == 1) return "Película";
    if (tipo == 2) return "Serie";
    if (tipo == 3) return "Documental";
    return "Evento en vivo";
}

```

```
static string ObtenerTextoClasif(int clasif)
{
    if (clasif == 1) return "Todo público";
    if (clasif == 2) return "+13";
    return "+18";
}

static string ObtenerTextoProduccion(int prod)
{
    if (prod == 1) return "Baja";
    if (prod == 2) return "Media";
    return "Alta";
}

static void Linea(char caracter, int largo)
{
    Console.WriteLine(new string(caracter, largo));
}
}
```